FlowGuard — Problem, Solution & Unique Approach

The Problem

On September 23-24, 2023, Nagpur flooded. Ambazari Lake overflowed into the Nag River, which was already choked by blocked drains, encroachment, and silt. 4 people died, 400+ were evacuated, roughly 10,000 houses were affected. A Public Interest Litigation (PIL) is active, demanding ₹2,000 crore for Nag River rejuvenation and holding NMC/ Maha Metro accountable for blocking natural water channels.

The structural gap: every flood-warning system that exists today — including anything proposed for Nagpur — uses a single water-level sensor at one point, with a fixed threshold, that fires an alert once the water is already high. This tells you a flood is happening. **It cannot tell you which specific drain is blocked, or that it was losing capacity for weeks before the flood, because nobody is measuring that.**

Right now, the only way anyone finds out a drain is obstructed is a manual survey team physically inspecting it — rare, slow, and easy to skip.

Our Solution

FlowGuard continuously answers a different, more useful question: **not "is it flooding," but "which specific drain segment has silently lost capacity, and by how much, on an ordinary day, long before the next storm."**

The core mechanism — physics, not guesswork

We use the **orifice equation**, derived from Bernoulli's principle — a standard, century-old civil engineering formula that relates how fast water drains out of an opening to how deep the water is above it:

```
Q = Cd × A × √(2gh)
```

We know how much water is flowing in (Q). We measure how high the water gets (h). We solve backward for the channel's **real, current open area (A)**. If that calculated area is smaller than the drain's known clean opening — that gap **is** the blockage, quantified as a percentage, from data alone, without anyone inspecting anything physically.

Three layers of depth, not just one sensor

1. **Physics layer** — the orifice equation above, calibrated to each specific channel using a controlled test pour (removes the need to separately account for water's viscosity or friction — those effects are absorbed into one measured constant, Cd).
2. **ML confirmation layer** — a single noisy reading (a splash, a sensor glitch) shouldn't trigger a false alarm. We use an Isolation Forest trained on known-clean behavior to confirm that a *sustained pattern* of rising blockage is real, not noise — the same statistical discipline used in industrial predictive maintenance.
3. **Network cascade layer** — Nagpur's water bodies aren't independent; Ambazari Lake feeds the Nag River, which flows downstream through multiple segments. We model this as a graph and use **Muskingum flow routing** (a standard hydrological technique) to predict how a rainfall pulse propagates and attenuates through the network — giving genuine multi-hour lead time to downstream neighborhoods, not just a point-in-time alert.

What makes this genuinely unaddressed

Every competing approach treats flood monitoring and infrastructure auditing as two separate problems — a sensor that alarms during a

flood, and a separate, manual, rare inspection process for finding blockages. **We treat them as the same data problem**: the gap between what physics predicts and what the sensor observes *is* the blockage, continuously, automatically, on any ordinary day — before the next storm, not during it.

The Demo

One real physical hardware node (ESP32 + ultrasonic sensor + a small model drain channel) proves the core physics and ML detection genuinely work — we pour water, show clean-channel behavior, then physically insert an obstruction and show the system catch the discrepancy live, in real time. The rest of Nagpur's water network (other lakes, drain segments) is completed in software using the same real math, showing how this scales citywide.

One-line pitch

"Nagpur already knows blocked drains caused a deadly flood — there's a court case about it right now. FlowGuard doesn't just warn you next time; it finds the exact blocked segments automatically, using physics and machine learning, before the next flood happens — not after."